

Nexus: por que federation importa** Três razões concretas: **a) Compliance multi-jurisdicional:** - Cliente na Europa: dados na UE. - Cliente no Brasil: dados no Brasil. - Mas a análise pode envolver modelos em ambos. -

Federation resolve: **dados ficam em casa, lógica viaja**. **b) Latência local:** - Afiliado em Tokyo não pode esperar 300ms para resposta que atravessa Atlântico. - Cada região tem seu nó Nexus. - Federation coordena sem adicionar latência significativa. **c) Resiliência sistêmica:** - Se nó EU cai, nó BR assume parte da carga (com degradação). - Sem federation, outage regional = outage global. **4. Arquitetura técnica do Federation Gateway** ``

Federation Gateway | | | — Registry (lista de nós, capabilities) | | | — Router (decide qual nó para cada req) | | | — Translator (formato, schema, locale) | | | — Trust manager (auth, certs) | | | — Audit logger (cross-node logs) | | | — Cost tracker (custo por delegação) |

| `` Implementado em Rust para performance. Expõe **API interna** que cada nó Nexus consome.

5. Protocolo A2A (Agent-to-Agent) em produção O protocolo A2A usado pela Nexus: ``yaml message_id: uuid sender_node: nexus://br.sao/

instance-42 receiver_node: nexus://eu.frankfurt/instance-7 performative:

REQUEST content: action: "analyze_cohort" args: cohort_id: "abc-123"

preserve_pii: true constraints: max_latency_ms: 5000 data_residency: EU

context_ref: "conv_xyz" signature: ed25519 `` **Características:** -

Assinatura digital: cada mensagem é assinada. - **Contract implícito:**

cada nó declara o que aceita. - **Idempotência:** replay-safe. - **Tracing

distribuído:** cada hop tem trace_id. **6. PII-gating e data residency**

Regra de ouro: dados PII **nunca** atravessam fronteira sem consentimento. Como funciona na prática: - **Cohort brasileira** (PII de brasileiros) → analisada em nó BR. - Se a análise precisar de modelo maior (e.g., GPT-4 hospedado nos EUA): - Dados são **anonimizados** localmente.

- Só dados não-PII são enviados. - Resultado volta ao nó BR. - Tenant vê

apenas o resultado, nunca soube que houve federation. **Em código:**

``python def federated_analyze(cohort_id, target_node): data = load_cohort(cohort_id) safe_data = strip_pii(data) # local result =

target_node.analyze(safe_data) # remote enriched_result =

attach_local_context(result, data) # local return enriched_result `` **7. Live

demo: agente brasileiro + agente europeu + agente asiático**

Demonstração ao vivo: 1. Afiliado brasileiro pede análise de cohort de 5.000 contatos. 2. Orchestrator BR identifica que precisa de modelo de linguagem avançado. 3. Delega para nó UE (que tem Claude Opus hospedado). 4. Nó UE recebe apenas dados anonimizados (sem PII). 5. Retorna análise em <2s.

6. Nó BR re-contextualiza com PII local. 7. Afiliado recebe resultado

completo. Tempo total: ~3.5s. Custo: \$0.08. Compliance: 100%. **8.

Latência, custo e resiliência em federation** **Latência:** - Hop local:

50-200ms. - Hop federation: 200-500ms adicional. - Caching agressivo reduz para <100ms em hits. **Custo:** - Cada hop federation: \$0.001-0.01. -

Tracking granular por tenant. **Resiliência:** - Se nó UE indisponível,

fallback para modelo BR. - Degradação graceful: latency sobe, mas funciona.

- SHO monitora federation health com sensores específicos. **9. Casos reais

(3 white-labels federados)** **Caso 1 — BancoX:** - Operação em 5 países

(BR, MX, ES, PT, US). - Federation reduziu latência média de 800ms para

120ms. - Compliance automatizado por residência. **Caso 2 —

OperadoraY:** - 8 jurisdições latam. - Dados pessoais em cada país. - Análise

cross-region permitida em dados anonimizados. - Resultado: LTV calculado em tempo real. **Caso 3 — AgênciaGlobal:** - Operação em 3 continentes. - Modelo grande centralizado em 1 região. - Federation Gateway gerencia fila de tarefas. - Custo reduzido em 40% vs. deploy local. **10. Quando NÃO usar federation (anti-patterns)** **NÃO use federation quando:** - Tudo é local e cabe num nó. - Não há requisito multi-jurisdicional. - Latência adicional é inaceitável (<200ms). - Compliance não exige separação. **Use single-node até federation ser claramente benéfica.** **11. Roadmap técnico 2026-2027** **Q3 2026:** - Federation Gateway GA (general availability). - A2A protocol v1.0 estabilizado. - Suporte a 5 regiões iniciais. **Q4 2026:** - Shared context (Nível 4) em beta. - Federation para SMB (não só enterprise). - Marketplace de nós federados. **Q1 2027:** - CRDT-based shared memory. - Federation cross-cloud (AWS + GCP + Azure). - 15+ regiões disponíveis. **12. Q&A** Perguntas esperadas: - Como funciona quando tenant não tem nó próprio? - Qual o custo real de federation? - Como testar federation em desenvolvimento? - Federation quebra LGPD? **13. Materiais de estudo** - **Apostila 11** — SHO em Produção - **Apostila 12** — IOAID — Arquitetura Profunda - **Apostila 14** — Multi-Tenant & White-Label - [Lib-Nexus/knowledge-base/01-modelo-ioaid.md](../Lib-Nexus/knowledge-base/01-modelo-ioaid.md) - [fase8/](../fase8/) — código de federation --- *Nexus Affil'IA'te · Academ'IA · Webinar WB-2026-08 · v1.0.0 · Setembro 2026*